
Redes Neurais e Aprendizagem Profunda

— Ajuste de redes neuronais —

Sumário

Número de camadas escondidas

Número de neurónios por
camada

Outros hiperparâmetros

Keras

Ajuste de hiper-parâmetros

- Muitos hiperparâmetros para afinar
 - A flexibilidade das redes neurais é também o seu inconveniente!
- O que afinar?
 - arquiteturas inimagináveis
 - MLP simples
 - nº camadas
 - nº neurónios por camada
 - função de ativação em cada camada
 - inicialização dos pesos
 - ...
- Como fazer?
 - testar várias combinações e ver quais funcionam melhor
 - no conj de validação
 - usar validação cruzada k-pastas
 - scikit-learn
 - `GridSearchCV()`
 - `RandomizedSearchCV()`
- Tempo necessário depende de
 - hardware
 - tamanho do conj
 - complexidade do modelo
 - nº iterações
 - nº de pastas

Pesquisa do "melhor" modelo

- Pesquisa aleatória
 - é fácil de usar
 - funciona bem para muitos problemas razoavelmente simples
- aplicar *zooming*
 - correr uma pesquisa aleatória rápida usando intervalos largos
 - correr nova pesquisa usando intervalos mais apertados centrados nos melhores valores obtidos

Bibliotecas Python para otimização de hiperparâmetros

- *Hyperopt*
 - Biblioteca popular para otimizar todos os tipos de espaços de pesquisa complexos
 - inclui valores reais, como a taxa de aprendizagem e valores discretos, como o número de camadas
- *Hyperas, kopt ou Talos*
 - Bibliotecas úteis para otimizar hiperparâmetros para modelos Keras
 - os dois primeiros são baseados no Hyperopt
- *Keras Tuner*
 - Biblioteca fácil de usar desenvolvida pela Google para modelos Keras
 - serviço hospedado para visualização e análise
- *Spearmint*
 - Biblioteca de otimização bayesiana.
- *Scikit-Optimize (skopt)*
 - Biblioteca de uso geral
 - BayesSearchCV() executa a otimização bayesiana com interface semelhante a GridSearchCV()
- *Hyperband*
 - Biblioteca de ajuste rápido de hiperparâmetros baseada no artigo Hyperband de Lisha Li et al.
- *Sklearn-Deap*
 - Biblioteca baseada em algoritmos evolucionários, com interface do tipo GridSearchCV()

Otimização e algoritmos evolucionários

- DeepMind
 - *Population Based Training of Neural Networks*, (2017). arXiv:1711.09846
 - otimização de modelos e hiperparâmetros
- Google
 - AutoML
 - pesquisa de hiperparâmetros
 - pesquisa de arquiteturas
- Uber
 - Deep Neuroevolution
 - treino de redes sem descida do gradiente

Número de camadas escondidas

Número de camadas escondidas

- Para muitos problemas **1 camada** é suficiente!
 - teoricamente permite modelar qualquer função desde que tenha neurónios suficientes!
- Problemas complexos
 - redes profundas têm uma eficiência de parâmetros muito maior
 - permitem a utilização de um nº de neurónios exponencialmente menor!
 - obtêm melhores desempenhos com a mesma quantidade de dados de treino...

Problemas e redes profundas

- Problemas reais
 - dados estruturados de forma hierárquica
 - floresta → árvores → ramos → folhas
- Redes profundas utilizam esta característica
 - camadas mais baixas: modelam estruturas de baixo nível
 - segmentos de linha com várias formas e orientações
 - camadas intermédias: combinam aquelas para modelar estruturas de nível intermédio
 - quadrados, círculos
 - camadas mais altas: combinam aquelas para modelar estruturas de alto nível
 - faces

Arquitetura hierárquica

- Permite uma convergência mais rápida para uma boa solução
- Melhora a capacidade de generalizar para outros conjuntos
 - Exemplo
 - um modelo de reconhecimento de faces
 - pode ser usado usado para reconhecer estilos de cabelo
 - reutilizando as camadas mais baixas do modelo inicial
- Transfer learning
 - a rede não precisa aprender do início todas as estruturas de baixo nível de problemas semelhantes

Sumário

- Maioria dos problemas
 - 1 ou 2 camadas escondidas
- Problemas complexos
 - usar redes com n° de camadas crescentes até sobre-ajustar
- Problemas muito complexos
 - dezenas de camadas ou até centenas (sem estarem completamente ligadas)
 - necessária uma quantidade enorme de dados
 - raramente necessário treinar de raíz
 - comum usar partes de uma rede treinada que executa tarefas semelhantes
 - treino muito mais rápido
 - necessários muito menos dados

Número de neurónios por camada

Número de neurónios por camada

- Camadas de entrada e saída
 - número determinado pelo tipo de entrada e saída da tarefa
- Camadas escondidas
 - comum usar números em pirâmide
 - muitas estruturas de baixo nível fundem-se em menos estruturas de alto nível
 - mesmo nº de neurónios em todas as camadas
 - em muitos casos, o produz desempenho igual ou até melhor
 - existe apenas um hiperparâmetro para afinar (em vez de um por camada)
 - *prática abandonada (nº de parâmetros a treinar é maior)*

Estratégias

- Básica
 - aumentar o nº de neurónios gradualmente até sobre-ajustar
- Mais simples e eficiente
 - escolher um modelo com mais camadas e neurónios
 - usar *early-stopping* ou outra técnica de regularização para prevenir sobre-ajustamento
- Em geral
 - melhor aumentar o nº de camadas que o nº de neurónios por camada

Outros hiperparâmetros

Taxa de aprendizagem

- Possivelmente o parâmetro mais importante
 - geralmente o valor ótimo é cerca de metade do valor máximo (aquele em que o algoritmo diverge)
- Estratégia
 - treinar o modelo por algumas centenas de iterações com uma taxa pequena ($\sim 10^{-5}$)
 - aumentar gradualmente até um valor alto (~ 10)
 - multiplicar a taxa por um fator constante em cada iteração
 - exemplo: $\exp(\log(10^6) / 500) \rightarrow 10^{-5}$ a 10 em 500 iterações
 - observar a perda como função da taxa de aprendizagem
 - começa por cair
 - quando a taxa for demasiado alta, a perda volta a subir
 - valor ótimo é um pouco inferior a esse valor (tipicamente 10x menor)
 - reinicializar o modelo e treiná-lo com esse valor

Tamanho do batch

LeCun: "*Friends don't let friends use mini-batches larger than 32!*"

- Grande impacto no desempenho e tempo de treino
- Valor grande
 - Benefício
 - GPUs processam os dados de forma mais eficiente
 - recomendação: usar o maior valor que "cabe" na RAM do GPU
 - Problema
 - muitas vezes provocam instabilidade no treino (especialmente no início)
 - pode não generalizar tão bem como com o uso de batches pequenos
- Conclusão
 - *Masters et al*: batches pequenos (2 a 32) geram melhores modelos em menos tempo
 - *Hoffer et al, Goyal et al*: é possível usar batches grandes (até 8192) c/ técnicas adicionais (warm up da taxa de aprendizagem)

Outros hiperparâmetros

Função de ativação

- Camadas escondidas
 - ReLU é uma boa aposta
- Camada de saída
 - depende do problema

Número de épocas

- não é necessário afinar
- usar *early stopping*

Otimizador

- muito importante
- discutido em aula posterior

Tensorflow, PyTorch, Keras

TensorFlow



- <https://www.tensorflow.org/>
- Desenvolvido pela Google Brain (para uso interno em investigação e produção)
 - Fev 2015: lançamento
 - Set 2019: TensorFlow 2.0
 - Mar 2024: r2.16
- Disponibilidade
 - Diversas plataformas: CPUs, GPUs, TPUs
 - desktops, clusters de servidores, dispositivos edge

PyTorch



- <https://pytorch.org/>
- Desenvolvido pela Meta AI (Facebook)
 - Set 2016: lançamento
 - Mar 2018: incorpora Caffe2 (Convolutional Architecture for Fast Feature Embedding)
 - Set 2022: mantido por PyTorch foundation (Linux Foundation)
 - Mar 2023: PyTorch 2.0
 - Jan 2024: r2.2

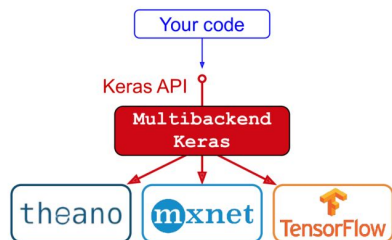
Keras

- <https://keras.io>
- Desenvolvido por François Chollet
 - Mar 2015: lançamento
 - Mai 2017: Keras 2.0
 - Nov 2023: Keras 3.0
 - Mar 2024: r3.1
- API de alto nível para aprendizagem profunda
 - Permite criar, treinar, avaliar, executar redes neuronais

Computation backends

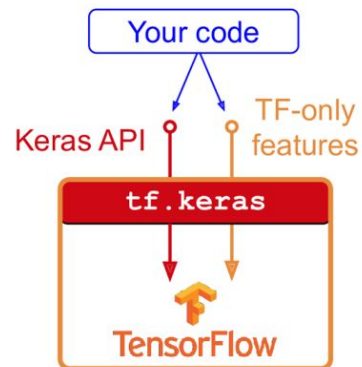
Multi-backend

- TensorFlow
- Até r2.3
 - CNTK
 - Theano
- A partir de r3.0
 - PyTorch
 - JAX



Tensorflow's Keras implementation

- `tf.keras`



Bibliografia

Capítulo 10

*Hands-On Machine Learning with
Scikit-Learn, Keras & TensorFlow*
